

Mobility in Smart Cities

Make Change

Practical Work — Exact Solving in C++

Ahmed Al-Muharaq
Owais Ahmed Khan

Academic year 2026–2027

Contents

1 Problem and objectives **1**

2 Design and implementation **2**

3 Task A: greedy ordered and unordered lists **2**

3.1 Algorithm and observation 2

3.2 Complexity experiment 3

4 Task B: enumerate every valid combination **3**

4.1 Recursive and iterative methods 3

4.2 Growth and interpretation 3

5 Task C: improving solutions, objective and cut **4**

6 Task D: heterogeneous limits **5**

7 Task E: three diverse feasible solutions **5**

8 Task F: explicit tabu list **6**

9 Validation, limitations and discussion **6**

10 Conclusion **7**

1 Problem and objectives

The *make change* problem asks for a set of coins whose total is exactly a requested amount. This practical work progresses from a fast greedy choice to exact enumeration and optimization, and then to limited stocks and alternative solutions. The course materials ask for iterative and recursive enumerations, an improving-solution trace, and a cut that avoids branches unable to improve the current best result [1, 2]. Tasks A–F extend this sequence to unordered input, heterogeneous limits, multiple diverse solutions and an explicit tabu set.

Our worked example uses integer cents throughout:

$$D = (500, 200, 100, 50, 20, 10, 5), \qquad M = 1235. \tag{1}$$

A solution is a nonnegative count vector $x = (x_1, \ldots, x_7)$ satisfying

$$\sum_{i=1}^{7} d_i x_i = M, \qquad x_i \in \mathbb{Z}_{\geq 0}. \tag{2}$$

Its cost (or fitness to minimize) is the total number of coins,

$$f(x) = \sum_{i=1}^{7} x_i. \tag{3}$$

For finite stock u_i, we add $x_i \leq u_i$. All seven values and all results below refer to the same example unless another target is stated.

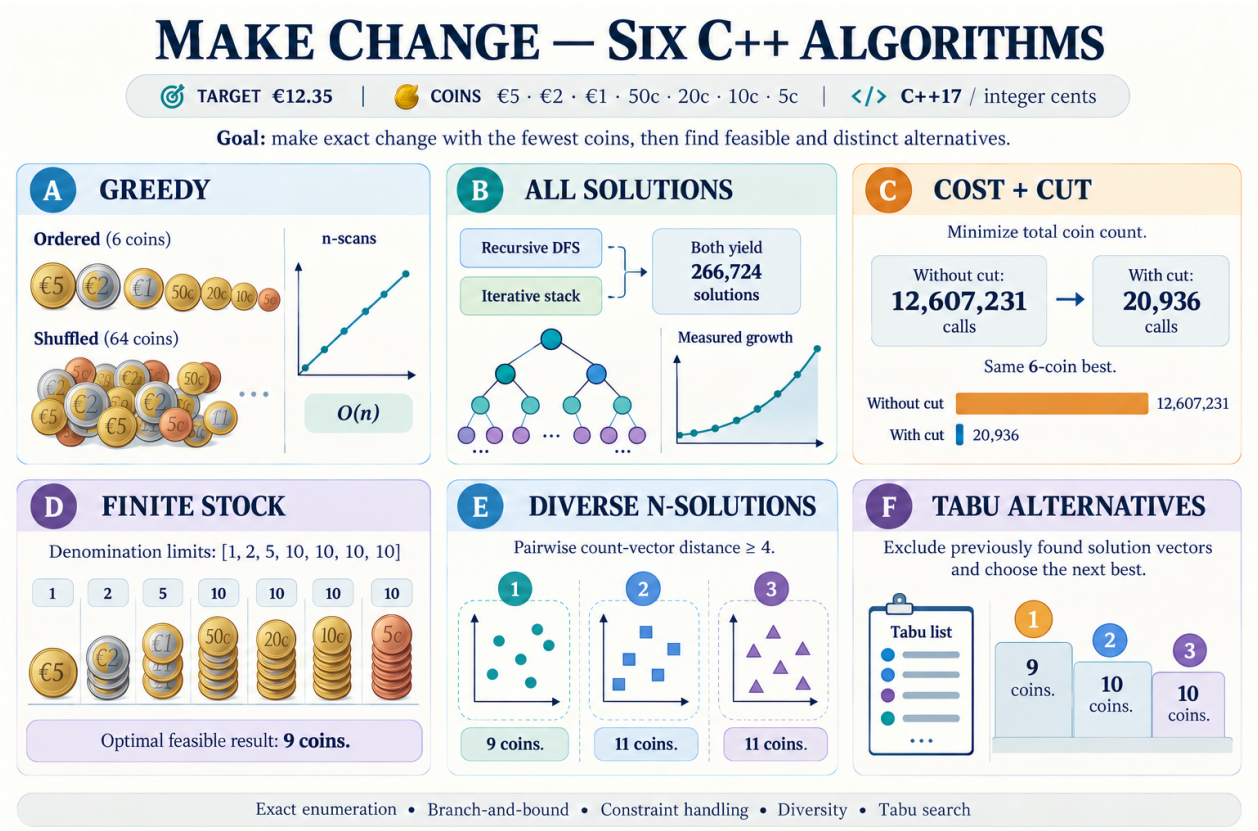


Figure 1: Visual overview of the six implemented tasks. Coin drawings and small decorative curves are schematic; the numeric labels are the program's observed results. Exact count vectors and benchmark data are given below.

2 Design and implementation

The program is a single C++17 file, `code/main.cpp`. The class `MakeChangeSolver` holds the denomination list and implements each task. The constructor sorts denominations in descending order. Monetary amounts are integers in cents, so equality checks and remainders do not depend on floating-point rounding. Counts are indexed in the same order as the denominations in Equation (1).

Table 1: Algorithm progression and generated evidence.

Task	Method	Evidence
A	One greedy scan, ordered or seeded shuffled input	`task_a_*.csv/svg`
B	Recursive depth-first search; explicit-stack iterative search	Two solution CSVs; `task_b_*.csv/svg`
C	Improving incumbents, with and without a lower-bound cut	Two improvement CSVs and call counts
D	Branch-and-bound subject to individual stock limits	Feasible and infeasible examples
E	Repeated exact search with pairwise distance threshold	Three diverse solutions
F	Repeated exact search with explicit forbidden vectors	Three ranked non-tabu solutions

Enumeration representation. At recursion level i, `counts[i]` specifies the number of coins d_i. The remaining amount is $r = M - \sum_{j<i} d_j x_j$. The procedure tries every legal count for the current denomination, updates r, and backtracks. A complete solution is emitted as soon as $r = 0$; since later counts are zero, each vector is emitted once. The iterative version replaces the call stack with frames `(index, remaining, next count)`. Its order matches the recursive descending-count traversal exactly.

Files and timings. Both full enumerations stream rows to CSV instead of holding 266,724 vectors in memory. Their reported durations include file writing. The Task C comparison writes only new incumbents in both variants; its timing excludes the later printing of the best vector. Microsecond measurements vary between runs and are not a proof of complexity.

3 Task A: greedy ordered and unordered lists

3.1 Algorithm and observation

For each denomination in the chosen order, take $q = \lfloor r/d_i \rfloor$ coins and update $r \leftarrow r \bmod d_i$. An exact solution exists for that particular greedy run when its final remainder is zero. The unordered test uses a fixed shuffle seed (42), making the example reproducible. The program performs the same one-pass rule on both orders; the seeded shuffle itself also takes linear time.

Table 2: Greedy results for €12.35; vector entries follow D.

Input order	Count vector in denomination order	Coins
Descending	$(2, 1, 0, 0, 1, 1, 1)$	6
Fixed shuffle $20, 500, 100, 50, 5, 10, 200$	$(0, 0, 0, 0, 61, 0, 3)$	64

The first vector is $2(500) + 200 + 20 + 10 + 5 = 1235$; the second is $61(20) + 3(5) = 1235$. Thus input order strongly affects the selected solution. Even descending order does *not* guarantee optimality for every denomination system: with $(4, 3, 1)$ and target 6, greedy uses $(4, 1, 1)$ (three coins), while $(3, 3)$ uses two.

3.2 Complexity experiment

The program benchmarks lists of $n \in \{10, 25, 50, 100, 200, 400, 800\}$ denominations and counts one scan for each denomination. Both orders perform exactly n scans. This deterministic operation count demonstrates $\Theta(n)$ for the selection loop; one-shot timings of the seven-coin example round to $0\,\mu$s and are unsuitable for fitting a timing curve.

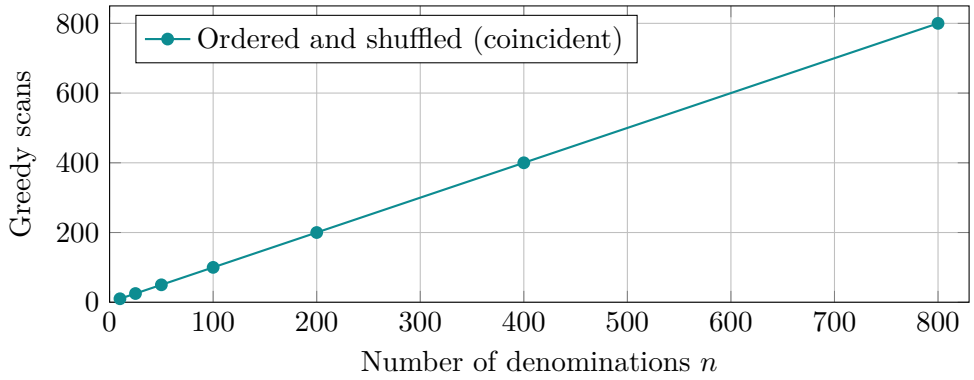


Figure 2: Task A: actual scan counts from `task_a_complexity.csv`; the two lines coincide.

4 Task B: enumerate every valid combination

4.1 Recursive and iterative methods

The recursive method descends from the maximum legal count of each denomination to zero and writes a vector whenever the remainder becomes zero. The iterative method uses explicit stack frames to reproduce exactly the same traversal and CSV order. This is exhaustive *backtracking*, not memoized dynamic programming: caching a subproblem cannot by itself avoid emitting all distinct solution vectors.

Exact comparison

For $M = 1235$, both methods generate 266,724 solutions. The two CSV files match byte for byte. Recursive search makes 12,607,231 calls. On the user's Windows run, recursive enumeration plus CSV output took 110,906 μs; the explicit-stack version plus CSV output took 75,353 μs. These are single-run observations, not a general speed ranking.

The list is sorted, so each denomination has one count position and each unique combination has one representation. To validate the output, we checked that every one of the 266,724 rows satisfies Equation (2), and compared the recursive and iterative files byte for byte.

4.2 Growth and interpretation

A count-only benchmark (no solution-file writes) measures recursive calls at several target amounts. In the exact example, the target has 11 bits when encoded as a binary integer. The plot uses bit length on the horizontal axis, with targets shown in the table. A fixed set of denominations and a growing amount have an enumeration cost that is polynomial in the *numeric amount* but may be exponential in its *binary input length*; more generally, output size is an unavoidable lower bound on enumeration time. The measured points illustrate rapid growth, but six observations do not prove an asymptotic bound.

Table 3: Task B count-only benchmark from the program (call counts are deterministic).

Target (cents)	Input bits	Valid vectors	Recursive calls
50	6	13	74
100	7	50	397
200	8	293	3,572
400	9	2,728	53,392
800	10	39,857	1,314,431
1,235	11	266,724	12,607,231

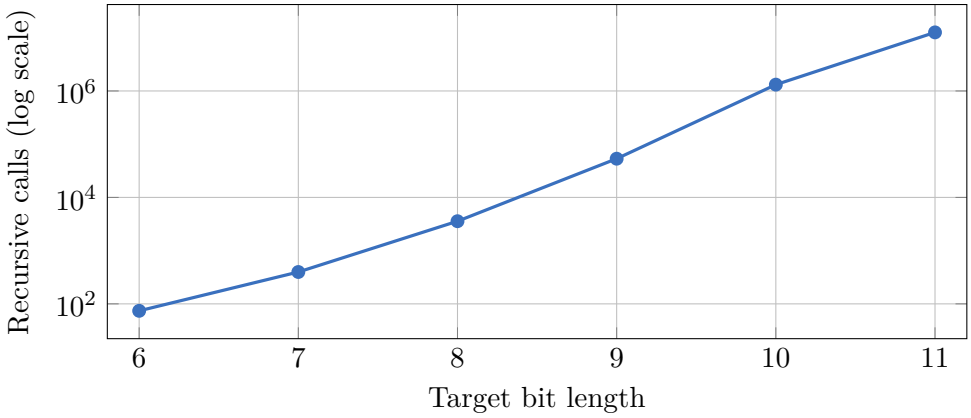


Figure 3: Task B: measured recursive calls versus binary target length, shown with a logarithmic vertical scale.

5 Task C: improving solutions, objective and cut

The search assigns cost $f(x)$ from Equation (3). It traverses counts in increasing order in this task, making successive improvements observable. A candidate is written to its improvement CSV *only* if its cost is strictly less than the current incumbent. Both the uncut and cut variants use the same traversal order and write the same 242 incumbent vectors.

With remaining amount r at index i and partial coin count c, the largest still available denomination is d_i. Even in the optimistic case, at least $\lceil r/d_i \rceil$ more coins are required. Hence the safe lower bound is

$$LB = c + \left\lceil \frac{r}{d_i} \right\rceil . \tag{4}$$

If $LB \geq f(x_{\text{best}})$, no completion of that branch can *strictly* improve the incumbent, so the branch is cut. This bound deliberately ignores coin stock and can only underestimate the actual additional cost. A cut preserves the minimum cost because it discards only branches provably unable to beat the current best.

Table 4: Task C: same optimum and improvement trace, much smaller search. Timings are from the user's example run.

Variant	Calls	Cut nodes	Improvements	Time (μs)
Without cut	12,607,231	0	242	51,370
With cut	20,936	4,731	242	187

The best vector is $(2, 1, 0, 0, 1, 1, 1)$, costing six coins. The cut makes about 602 times fewer calls in this example. The measured times differ by about 275 times on this run. Relative to printing all

266,724 solutions, printing only 242 improvements reduces output rows by about 99.91%; adding the cut reduces traversal, while the improvement trace remains the same.

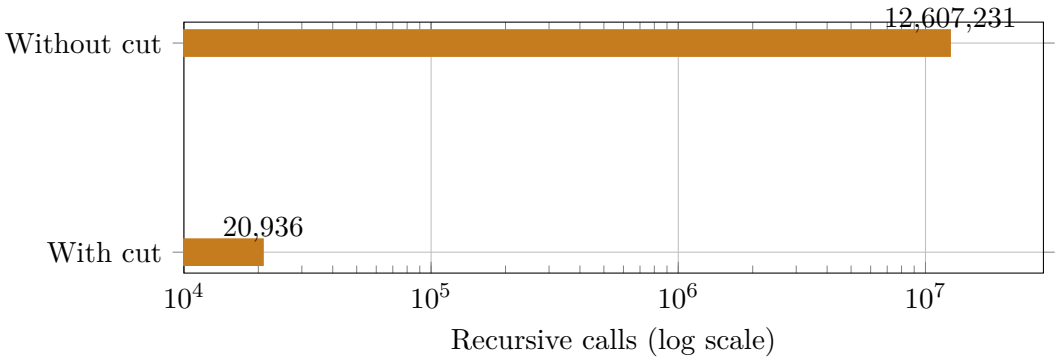


Figure 4: Task C: calls with and without pruning. The horizontal axis is logarithmic.

6 Task D: heterogeneous limits

An unlimited supply is no longer assumed. The test uses

$$u = (1, 2, 5, 10, 10, 10, 10), \qquad 0 \le x_i \le u_i. \tag{5}$$

At index i, the search tries at most $\min(\lfloor r/d_i \rfloor, u_i)$ coins. It uses the same safe optimistic bound to prune branches that cannot lower the current best. The optimal feasible result is

$$x_D = (1, 2, 3, 0, 1, 1, 1), \quad \sum_i d_i x_{D,i} = 1235, \quad f(x_D) = 9. \tag{6}$$

The search made 30 calls and cut 17 nodes on this example. With all limits set to zero and a positive target, the program correctly reports that no feasible solution exists. The unconstrained six-coin optimum is forbidden here because it requires two €5 coins but $u_1 = 1$.

7 Task E: three diverse feasible solutions

Task E asks for several solutions that are not too similar. We define a distance between two count vectors,

$$\Delta(x, y) = \sum_{i=1}^{7} |x_i - y_i|, \tag{7}$$

and choose $N = 3$ with threshold $\delta = 4$. At each iteration, exact constrained branch-and-bound finds the least-cost feasible vector whose distance from *each* already selected vector is at least four. Thus every chosen alternative is optimal *given the earlier selections and the threshold*; this sequential rule does not claim to find a globally best set of three vectors under a collective objective.

Table 5: Task E: sequentially selected count vectors; order of coordinates follows Equation (1).

Rank	Vector	Coins	Distance from earlier choices
1	$(1, 2, 3, 0, 1, 1, 1)$	9	—
2	$(1, 2, 3, 0, 0, 2, 3)$	11	$\Delta(2, 1) = 4$
3	$(1, 2, 2, 2, 1, 0, 3)$	11	$\Delta(3, 1) = 6$, $\Delta(3, 2) = 6$

The distance threshold is a user-defined diversity policy, not a claim that all coin-count differences matter equally in everyday use. Raising it can reduce the number of obtainable solutions or force a higher coin cost.

8 Task F: explicit tabu list

Task F uses a different rule: store complete previously returned vectors in a tabu set T and solve

$$\min f(x) \quad \text{subject to} \quad \sum_i d_i x_i = M, \quad 0 \le x_i \le u_i, \quad x \notin T. \tag{8}$$

The full-vector inequality $x \notin T$ allows solutions that are very similar to a previous one, provided they are not identical. After returning a result, the method inserts it into T and repeats. It therefore returns the best remaining cost at each step, subject to tie order in the search.

Table 6: Task F: successive best vectors outside the growing tabu set.

Rank	Vector	Coins
1	$(1, 2, 3, 0, 1, 1, 1)$	9
2	$(1, 2, 3, 0, 1, 0, 3)$	10
3	$(1, 2, 3, 0, 0, 3, 1)$	10

This method excludes exact duplicates and is not the same as Task E's distance threshold. The two 10-coin solutions in Task F are valid equal-cost alternatives.

9 Validation, limitations and discussion

Table 7: Checks performed on the submitted C++17 implementation.

Check	Method	Outcome
Build	`g++ -std=c++17 -O2 -Wall -Wextra`	No warnings
A totals	Sum coins in both printed vectors	Both equal 1,235 cents
B coverage	Validate all CSV rows against Equation (2)	266,724 valid rows
B agreement	Compare recursive and iterative CSV files bytewise	Identical
C preservation	Compare improvement CSVs bytewise	Identical 242 rows
D–F optima	Independent exhaustive enumeration under stock limits	Costs and distinctness confirmed
Infeasibility	Set all stock limits to zero	No solution

Implementation limits. The demonstration amount and coin denominations are currently defined in `main()`, and Task D's limits are supplied from `main()`. A different scenario requires changing these inputs and recompiling. Full enumeration generates a potentially large CSV and is unsuitable for unbounded targets. A direct greedy pass may fail to find an exact solution even when one exists, and a sorted list alone is insufficient to establish optimality. We therefore use exact enumeration and branch-and-bound when optimality matters.

Complexity summary. Task A scans n denominations and uses $O(n)$ counts. Unpruned enumeration has a branching search tree and must spend $\Omega(S)$ time to emit S solutions. A simple upper bound for the implemented recursion is $O(n \prod_{i=1}^{n} (\lfloor M/d_i \rfloor + 1))$ nodes; the actual tree is much smaller because residual amounts restrict later choices. Keeping only the current count vector needs $O(n)$ working space, excluding output buffers. The iterative enumerator also uses an explicit stack of depth at most n. The cut changes the number of explored nodes on the example, not the fact that worst-case exponential behavior remains possible. Task E and F perform a separate exact search for each requested alternative; their result checks add a comparison against earlier solutions.

10 Conclusion

The six tasks show how the same change-making instance evolves from a cheap, order-sensitive heuristic into exact enumeration and then exact optimization with pruning. For €12.35, enumeration finds 266,724 combinations; the minimum without limits is six coins, while finite stock raises the minimum to nine. A pairwise distance threshold produces diverse alternatives (9, 11, 11 coins), whereas tabu exclusion gives the successive best distinct vectors (9, 10, 10 coins). The large reduction in Task C's calls is measured on this instance and explained by its admissible lower bound.

References

[1] P. Canalda, *Mobility in Smart Cities: Dynamic Transportation Exact Solving*, Part 1, course slides, 23 September 2025. "Makes change" and iterative/recursive work (pages 24–31).

[2] P. Canalda, *Mobility and Smart Cities*, Part 2, course slides, 23 September 2025. Practical programs 1–7bis: enumeration, improving solutions, and cuts.